Introduction

1.Introduction

1.1 Background

In recent years, the field of Natural Language Processing (NLP) has witnessed significant advancements, primarily driven by the evolution of neural architectures from recurrent models to attention-based frameworks. Earlier approaches such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks were limited in capturing long-range dependencies and were computationally inefficient for large-scale training. The introduction of the Transformer architecture, based on the concept of self-attention and multi-head attention mechanisms, marked a paradigm shift in NLP. This advancement enabled models to process entire sequences in parallel while effectively capturing contextual relationships, leading to substantial improvements in tasks such as machine translation, text generation, and question answering.

Building upon the Transformer architecture, Large Language Models (LLMs) such as LLaMA and Gemma have demonstrated remarkable capabilities across a wide range of language understanding and generation tasks. These models leverage large-scale pretraining on diverse corpora, followed by task-specific fine-tuning, resulting in highly versatile systems.

However, the majority of these models are trained on datasets that are predominantly English-centric, leading to limited performance in low-resource languages. Indic languages, including Bengali, Odia, and Gujarati, represent a substantial portion of the global population but remain significantly underrepresented in high-quality NLP datasets. Furthermore, these languages often suffer from a scarcity of labeled data, including question-answer pairs, classification datasets, and parallel corpora, which are essential for supervised learning and instruction tuning.

Although initiatives such as Sarvam AI and AI4Bharat have made progress in developing datasets and models for Indian languages, there remains a lack of extensive experimentation and benchmarking specifically focused on low-resource Indic languages. The challenges are further compounded in rural and less digitally represented regions, where linguistic diversity is high but digital data availability is limited.

The development of effective language models for these languages is crucial for enabling inclusive technological growth. Improved NLP systems can significantly enhance access to knowledge, healthcare information, government services, and educational resources for native speakers. In this context, there is a growing interest in developing Small Language Models (SLMs) that are efficient, scalable, and specifically tailored to handle the linguistic and data constraints of low-resource Indic languages.

1.2 Motivation

Despite the rapid progress of Large Language Models (LLMs), their effectiveness remains uneven across languages, with a significant performance gap between high-resource languages like English and low-resource Indic languages such as Bengali, Odia, and Gujarati. This disparity is primarily driven by the lack of large-scale, high-quality datasets and the limited availability of labeled data, including question-answer pairs, classification datasets, and parallel corpora.

Existing models often struggle with Indic languages due to several challenges: noisy and unstructured web data, frequent code-mixing with other languages, and the complexity of Indic scripts. As a result, these models exhibit poor performance in tasks such as translation, question answering, and conversational understanding when applied to these languages. Furthermore, most current research and benchmarking efforts focus on widely spoken languages, leaving low-resource Indic languages relatively underexplored.

Another critical motivation stems from the growing need for localized AI systems. A large portion of the population in India relies on regional languages for accessing information, yet many digital platforms and AI tools do not adequately support these languages. This creates a barrier to accessing essential services such as healthcare information, educational resources, and government schemes. Bridging this gap requires models that are not only multilingual but also capable of understanding the linguistic and cultural nuances of specific regions.

In addition, training large-scale models from scratch is computationally expensive and often infeasible in academic settings. This motivates the exploration of efficient alternatives, such as Small Language Models (SLMs), which can be trained or adapted using limited computational resources while still achieving competitive performance through high-quality data and targeted training strategies.

Therefore, this project is motivated by the need to address the data scarcity and performance limitations in Indic NLP, while also contributing towards the development of scalable, efficient, and inclusive language technologies that can better serve diverse linguistic communities.

1.3 Objective

The primary objective of this project is to design and implement an end-to-end pipeline for developing a Small Language Model (SLM) tailored to low-resource Indic languages, specifically Bengali, Odia, and Gujarati. The focus of the project is to address challenges related to data scarcity, data quality, and efficient model adaptation.

The first objective is to collect and curate high-quality datasets from diverse sources. This includes leveraging curated datasets from platforms such as AIKosh and AI4Bharat, as well as extracting structured unlabelled data from sources like Wikipedia and news websites. Additionally, synthetic data generation techniques will be employed to augment limited labeled datasets, particularly for tasks such as question answering and machine translation.

The second objective is to develop a robust and scalable data processing pipeline. This involves performing language filtering to isolate target languages, cleaning and deduplicating raw text to remove noise and redundancy, and categorizing the processed data into distinct groups such as pretraining corpora, translation datasets, and instruction-based datasets. Special attention will be given to tokenization strategies, ensuring effective handling of Indic scripts using techniques such as SentencePiece or Byte Pair Encoding.

The third objective is to adapt existing pretrained language models, such as LLaMA and Gemma, to the target languages through continued pretraining on curated Indic datasets. This approach enables efficient utilization of existing model capabilities while enhancing performance on low-resource languages.

Finally, the project aims to perform instruction tuning using curated and synthetic datasets, including question-answer pairs, conversational data, and translation tasks. This stage is intended to improve the model's ability to follow instructions, generate coherent responses, and perform multilingual tasks effectively.

Overall, the objective is to build an efficient, scalable, and high-quality multilingual language model that can contribute to improved accessibility, knowledge dissemination, and language technology support for underrepresented Indic languages.

Related Works

2. Related Work (with Comparisons)

2.1 Introduction

The development of Small Language Models (SLMs) for Indic languages has gained significant attention due to the need for efficient, deployable, and linguistically specialized models. While large multilingual models such as LLaMA and Gemma demonstrate strong generalization capabilities, they often underperform on low-resource Indic languages like Bengali, Odia, and Gujarati.

This limitation arises due to:

- Insufficient high-quality Indic training data
- Inefficient tokenization for Indic scripts
- Lack of linguistic and cultural specialization

This section reviews these models and provides a comparative analysis.

2.2 Early Multilingual Encoder Models

IndicBERT

IndicBERT, developed by AI4Bharat, is an efficient multilingual model based on the ALBERT architecture. It is trained on approximately 9 billion tokens across 12 Indic languages using masked language modeling.

Key Features:

- Lightweight and parameter-efficient
- Designed for multilingual understanding
- Suitable for resource-constrained environments

Strengths:

- Strong performance on:
    - Text classification
    - Named Entity Recognition (NER)
    - Sentiment analysis
- Demonstrates efficiency without sacrificing much accuracy

Limitations:

- Encoder-only architecture:
    - Cannot perform text generation
- Limited contextual reasoning compared to modern LLMs
- Not suitable for conversational AI

MuRIL (Multilingual Representations for Indian Languages)

MuRIL, developed by Google, extends the BERT framework by incorporating parallel and transliterated data. It supports 17 Indian languages and is designed to improve cross-lingual understanding.

Key Features:

- Trained on:
    - Parallel corpora
    - Transliteration pairs
- Handles multilingual and code-mixed data

Strengths:

- Strong cross-lingual transfer learning
- Effective handling of:
    - Code-mixed text
    - Transliterated inputs
- Improved performance on low-resource languages

Limitations:

- Encoder-only:
    - No generative capability
- Larger and less efficient than IndicBERT
- Limited performance on long-form tasks

2.3 Small and Efficient Decoder-Based Models

Sarvam-1

Sarvam-1 is a 2-billion-parameter decoder-only language model designed specifically for Indic languages and trained on a large Indic-focused corpus.

Key Features:

- Decoder-based architecture
- Supports generative tasks
- Trained on large-scale Indic data

Strengths:

- Strong performance on:
    - Question answering
    - Translation
    - Conversational tasks
- Demonstrates effectiveness of data-centric training

Limitations:

- Performance varies across low-resource languages
- Limited public training details
- Dataset imbalance may affect consistency
-

Nemotron-4-Mini-Hindi-4B

Nemotron-4-Mini is a 4B parameter model optimized for efficiency using pruning and distillation techniques.

Key Features:

- Derived from a larger multilingual model
- Optimized for deployment
- Focused on Hindi-English tasks

Strengths:

- Suitable for:
    - On-device deployment
    - Low-resource environments
- Efficient inference speed
- Strong generation capabilities

Limitations:

- Primarily Hindi-focused
- Limited multilingual generalization
- Dependent on parent model quality

Pragna-1B

Pragna-1B is a lightweight multilingual model based on LLaMA-2 architecture with a tokenizer optimized for Indic scripts.

Key Features:

- ~1.25B parameters
- Custom tokenizer for Indic languages
- Supports Bengali, Gujarati, Hindi, English

Strengths:

- Reduced token fragmentation
- Improved efficiency for Indic scripts
- Open-source and flexible

Limitations:

- Limited reasoning ability due to smaller size
- Dependent on base architecture
- Limited large-scale benchmarking
-

<u>2.4 General Multilingual Models</u>

LLaMA and Gemma

These are general-purpose large language models trained on global datasets.

Strengths:

- Strong reasoning and generation capabilities
- Widely used architectures

Limitations:

- Poor Indic language representation
- Inefficient tokenization for Indic scripts
- Lack of cultural and linguistic alignment

mT5 and BLOOM

Large multilingual models supporting 100+ languages.

Strengths:

- Strong translation capabilities
- Broad language coverage

Limitations:

- High computational cost
- Not optimized for Indic-specific nuances

2.5 Comparative Analysis

Comparison of Indic and Multilingual Language Models

Model	Type	Languages	Generative	Indic Focus	Efficiency
IndicBERT	Encoder	~12	No	High	High
MuRIL	Encoder	~16	No	High	Medium
Sarvam-1	LLM	Indic	Yes	High	Medium
LLaMA	LLM	Global	Yes	Low	Medium
Gemma	LLM	Global	Yes	Low	High
mT5/BLOOM	LLM	100+	Yes	Medium	Low

(Figure: Comparative analysis of Indic and multilingual language models)

2.6 Key Observations

From the above models, the following trends are observed:

- Encoder-based models:
  - Efficient but non-generative
- Small LLMs:
  - Balanced but depend heavily on data quality
- Large models:
  - Powerful but inefficient and not specialized

2.7 Research Gaps

The major gaps identified are:

- Underrepresentation of:
  - Odia
  - Gujarati
  - Bengali
- Inefficient tokenization for Indic scripts
- Noisy and unstructured training datasets
- Lack of efficient generative models
- High computational requirements

Proposed Techniques

3. Proposed Technique(s) and Algorithm(s)

This section outlines the proposed methodology for building a Small Language Model (SLM) for low-resource Indic languages, namely Bengali, Odia, and Gujarati. The approach is structured as a multi-stage pipeline consisting of data collection, preprocessing, tokenization, and model training.

3.1 Data Collection Strategy

The data collection process integrates multiple heterogeneous sources to ensure both quality and diversity of linguistic data.

3.1.1 Curated and Government Datasets

High-quality datasets are collected from established platforms such as AIKosh, AI4Bharat, and Hugging Face. These datasets include monolingual corpora, parallel translation datasets, and task-specific labeled data. Additionally, the OSCAR Dataset is utilized to obtain large-scale web data that has already undergone preliminary language filtering.

These curated datasets form the backbone of the training corpus due to their relatively high quality and structured format.

3.1.2 Structured Unlabelled Data

To supplement curated datasets, structured unlabelled data is collected from reliable textual sources such as Wikipedia and regional news platforms, including Anandabazar Patrika (Bengali), Sambad (Odia), and Gujarat Samachar.

This data is particularly valuable for learning grammatical structure, writing style, and domain-specific vocabulary. Unlike web-scale crawled data, these sources are more consistent and linguistically rich.

3.1.3 Synthetic Data Generation

Given the scarcity of labeled datasets in low-resource languages, synthetic data generation is employed to augment training data. Strong pretrained models are used to generate:

- Question-answer pairs for instruction tuning
- Parallel translation data for multilingual alignment
- Conversational data for dialogue modeling

This approach helps in expanding labeled datasets, especially for tasks where manual annotation is limited or expensive.

3.2 Data Preprocessing Pipeline

A robust data preprocessing pipeline is designed to ensure high data quality before training.

3.2.1 Language Filtering

Language identification is performed using tools such as LangDetect to ensure that only target language data is retained. This step removes:

- Non-target language text
- Code-mixed sentences
- Irrelevant multilingual noise

This is particularly important for Indic datasets, where code-mixing with English and Hindi is common.

3.2.2 Data Cleaning and Deduplication

Raw datasets often contain noise such as HTML tags, advertisements, and repeated content. The cleaning process involves:

- Removal of HTML/XML tags
- Normalization of Unicode text
- Elimination of special characters and formatting artifacts

Deduplication algorithms are applied to remove redundant sentences and documents, ensuring that the model does not overfit on repeated patterns.

3.2.3 Data Categorization

The processed data is categorized into distinct subsets based on training objectives:

- Pretraining Corpus: Large-scale unlabelled text used for learning general language representations
- Instruction/QA Data: Structured input-output pairs for instruction tuning
- Translation Data: Parallel corpora for multilingual learning

This separation enables targeted training strategies for different tasks.

3.3 Tokenization Strategy

Tokenization plays a critical role in handling the morphological and script-level complexities of Indic languages.

Subword tokenization techniques such as SentencePiece and Byte Pair Encoding (BPE) are employed to segment text into meaningful units. These methods are particularly effective in handling:

- Rich morphology
- Rare and compound words
- Multiple scripts and Unicode representations

A custom tokenizer is trained on the combined multilingual corpus to ensure optimal vocabulary coverage across Bengali, Odia, and Gujarati.

3.4 Model Training Framework

The training process is implemented using OLMo, an open-source framework designed for efficient large-scale language model training.

Instead of training a model from scratch, the proposed approach leverages pretrained models such as LLaMA, Gemma, and models developed by Sarvam AI. These models provide a strong initialization, reducing computational cost and training time.

3.5 Training Methodology

The training process is divided into two primary stages:

3.5.1 Continued Pretraining

In this stage, the pretrained model is further trained on Indic-specific corpora, including datasets from OSCAR, AI4Bharat, and AIKosh. The objective is to adapt the model to the linguistic patterns, vocabulary, and syntax of the target languages.

This stage improves the model’s ability to understand and generate text in Bengali, Odia, and Gujarati.

3.5.2 Instruction Tuning

Following pretraining, the model undergoes instruction tuning using structured datasets. This includes:

- Question-answer datasets
- Machine translation pairs
- Synthetic conversational data

Instruction tuning enhances the model's ability to follow user prompts, generate contextually relevant responses, and perform multilingual tasks effectively.

3.6 Summary of Approach

The proposed methodology integrates high-quality data collection, rigorous preprocessing, and efficient training strategies to address the challenges of low-resource Indic languages. By combining curated datasets, synthetic data augmentation, and transfer learning from pretrained models, the approach aims to develop a scalable and effective Small Language Model tailored to regional linguistic needs.

Data Sets used

4. Data Set Used

4.1 Introduction

The performance of a language model heavily depends on the quality and diversity of the datasets used. For Indic languages such as Bengali, Odia, and Gujarati, high-quality data is limited and often noisy. Therefore, our approach focuses on combining curated datasets, web-based corpora, and synthetic data to build a robust training pipeline.

4.2 Primary Data Sources

AIKosh (IndiaAI Platform)
https://aikosh.indiaai.gov.in/home
A government-supported platform providing curated datasets for Indian languages. It will be used as a reliable source of structured and high-quality data.

Ganga-2-1B (Reference Model)
https://aikosh.indiaai.gov.in/home/models/details/ganga_2_1b.html
Used as a reference for dataset composition and training strategies for Indic language models.

AI4Bharat Datasets
https://ai4bharat.iitm.ac.in/datasets
A major source of multilingual Indic datasets, including parallel corpora useful for translation and cross-lingual learning.

Sangraha Dataset (HuggingFace)
https://huggingface.co/datasets/ai4bharat/sangraha
A large curated corpus for Indic languages, suitable for pretraining tasks.

4.3 Additional Data Sources

Web-Based Corpora

- Wikipedia (high-quality grammar)
- News sources:
    - Anandabazar Patrika (Bengali)
    - Sambad (Odia)
    - Gujarat Samachar

These sources help improve real-world language understanding and vocabulary coverage.

OSCAR Dataset
A large multilingual dataset used to supplement low-resource language data, though it requires cleaning.

4.4 Synthetic Data Generation

To overcome data scarcity, synthetic data will be generated using:

- Question-answer pair generation
- Translation-based augmentation

This helps improve performance on tasks like QA and instruction following.

Background Study

5. Background Study and Prerequisites

Before initiating the project, a strong foundation in deep learning, neural network architectures, and modern language models was developed. This background study was essential to understand the design and optimization of small language models, particularly for Indic languages.

Deep Learning Framework: PyTorch

The primary framework studied and used was PyTorch, which provides flexibility for building and training neural networks. It enabled hands-on understanding of model creation, tensor operations, backpropagation, and optimization techniques. PyTorch was also explored for implementing components of transformer-based architectures and experimenting with model behavior.

Core Neural Network Concepts

Several foundational neural network concepts were studied:

- Activation Functions: Functions such as ReLU and its variants were explored to understand how non-linearity is introduced into neural networks, allowing them to model complex patterns.
- Feedforward Neural Networks (FFN): These networks, consisting of linear transformations followed by non-linear activations, were studied as a key component of modern architectures. In transformers, FFNs are applied independently to each position to enhance representation learning.
- Convolutional Neural Networks (CNNs): CNNs were studied to understand spatial feature extraction and to contrast them with transformer-based models, which do not rely on convolution.

Attention Mechanism

A major focus of the study was the attention mechanism, which allows models to focus on relevant parts of the input sequence dynamically. Unlike traditional sequential models, attention enables capturing relationships between all tokens regardless of their position.

- Self-attention computes relationships between different words in the same sequence.
- It assigns weights to tokens based on their relevance to each other, producing context-aware representations.

This concept is fundamental to modern Natural Language Processing (NLP) systems.

Transformer Architecture

The Transformer model, introduced in the paper “Attention Is All You Need”, was studied in depth. This architecture replaces recurrence and convolution entirely with attention mechanisms.

Key components studied include:

- Multi-Head Attention: Allows the model to capture different types of relationships simultaneously.
- Encoder-Decoder Structure: The encoder processes input sequences, while the decoder generates outputs.
- Feedforward Layers: Applied after attention to transform representations.
- Positional Encoding: Introduced to retain sequence order information.

Transformers are highly parallelizable and can model long-range dependencies efficiently, making them the backbone of modern large language models.

Research Papers Studied

To gain deeper insights into the field, the following key research papers were studied:

- **Attention Is All You Need (Vaswani et al., 2017)**
  This paper introduced the Transformer architecture, demonstrating that attention mechanisms alone are sufficient for sequence modeling tasks. It laid the foundation for all modern LLMs

- **A Survey on Model Compression for Large Language Models**
  This paper provided an overview of techniques used to reduce model size and improve efficiency, such as pruning, quantization, and knowledge distillation. These techniques are particularly relevant for building small language models (SLMs) that can run on limited hardware.

Relevance to the Project

The knowledge gained from the above topics directly supports the project objectives:

- Understanding transformers enables working with modern LLM architectures.
- Knowledge of FFNs and attention helps in modifying and optimizing models.
- Model compression techniques are crucial for building efficient small-scale Indic language models.
- PyTorch provides the practical tools required for implementation and experimentation.

Experiments and Results

6. Experiment(s) and Result(s)

6.1 Relevant Papers Studied

1. "Language Models are Few-Shot Multilingual Learners"

- Authors: Tom B. Brown et al.
- Key idea:
  - LLMs can handle multiple languages even if trained mostly on English
  - You can reuse English capability for other languages

2. "On the Multilingual Capabilities of Large Language Models"

  - Studies how LLMs perform across languages
  - Performance drops significantly for low-resource languages
  - Translation-based approaches help

Based on this, the Translate-Then-Generate / Pivoting Approach was developed.
Input (low-resource) → Translate to English → LLM → Translate back

6.2 Pivoting Approach

We are developing a cross-lingual pipeline using LLMs to test the existing multi-lingual capabilities of LLMs. Factors involved:

- Translation (MT)
- LLM reasoning
- Back-translation

In this approach, input text in a target low-resource language is first translated into a high-resource language such as English. The translated input is then processed by a pretrained LLM, such as Gemma, which generates a response in English. Finally, the response is translated back into the original language.

This method leverages the strong reasoning and generation capabilities of LLMs in English while bypassing the need for extensive training data in low-resource languages. Prior research has demonstrated that LLMs exhibit significantly better performance in high-resource languages, and translation-based approaches can effectively bridge this gap. Such architectures are particularly useful for Indic languages, where labeled datasets and large-scale corpora are limited.

However, this approach introduces dependencies on the quality of machine translation systems, and errors in translation may propagate through the pipeline, affecting the final output. Despite

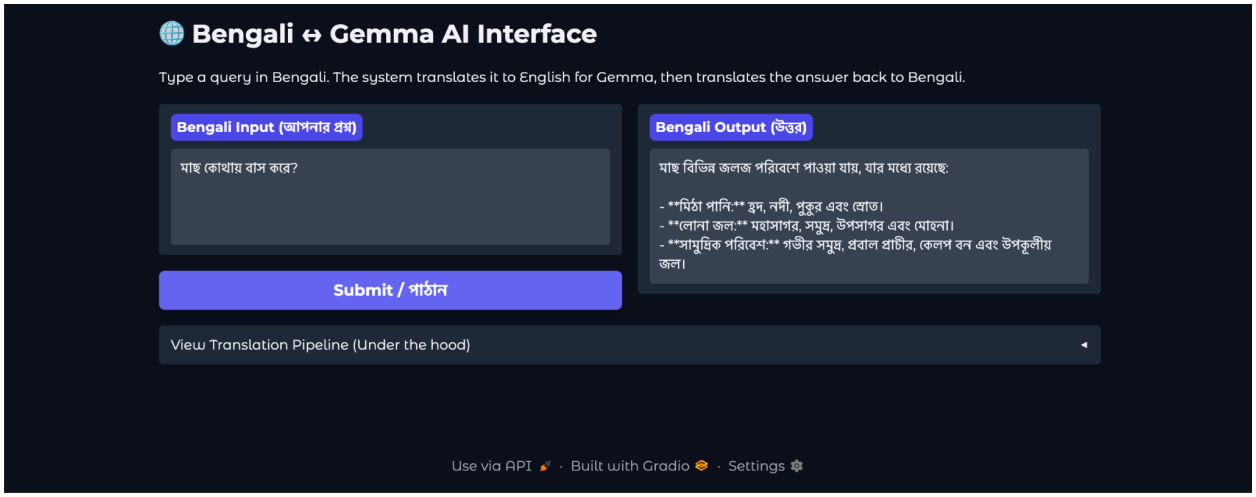
these limitations, translation-based LLM pipelines provide a scalable and efficient solution for enabling multilingual capabilities, especially in resource-constrained settings.

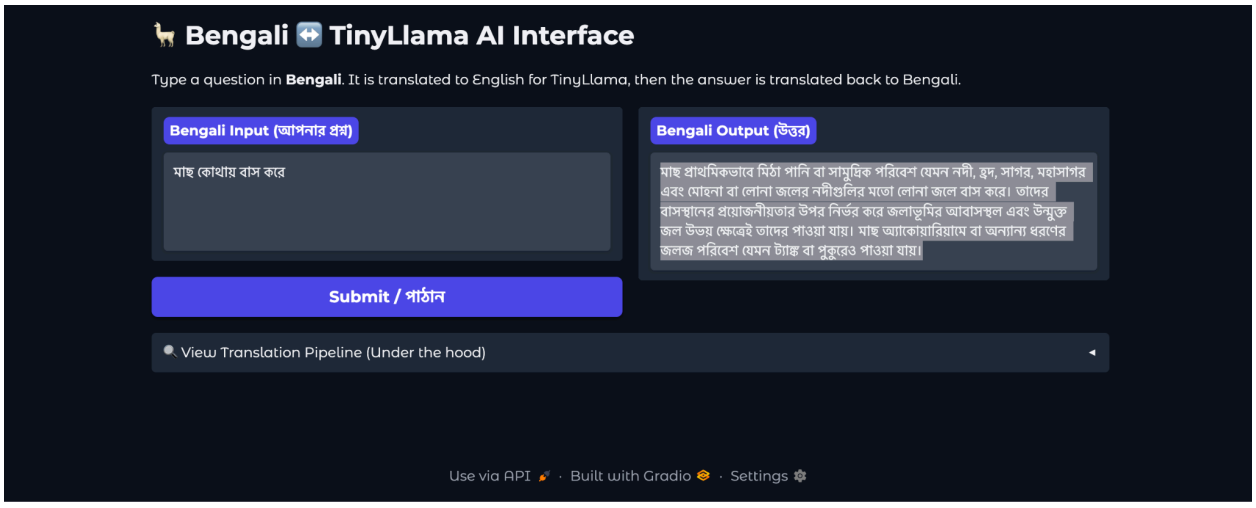
Advantages

- Works with low data
- Leverages powerful LLMs
- Easy to scale to new languages

Limitations

- Translation errors propagate
- Cultural nuance loss
- Latency (2 translations + inference)





6.3 Experimental Observations

The experimental results highlight several limitations of the proposed translation-based pipeline. A key observation is the relatively high latency of the system, which arises due to the multi-stage processing architecture. Each query undergoes multiple sequential steps, including translation from the source language to English, inference using the language model, and back-translation to the original language. This layered pipeline significantly increases response time compared to direct model inference.

Additionally, the system does not explicitly handle linguistic nuances present in Indic language queries. Since the model primarily operates in English, subtle contextual or cultural meanings in the original query may not be preserved during translation. As a result, the generated responses may lack accuracy in certain cases.

Another important issue observed is the occurrence of imperfect translations. Errors introduced during either forward or backward translation can propagate through the pipeline, leading to outputs that contain additional or unintended information. In some cases, the translated query deviates slightly from the original intent, causing the language model to generate responses that do not fully align with user expectations.

Overall, while the translation-based approach enables the use of powerful language models for low-resource languages, it introduces trade-offs in terms of latency, semantic fidelity, and response accuracy.

Conclusion and Future Work

7.1 Conclusion

This project explored the development of a Small Language Model (SLM) pipeline for low-resource Indic languages, specifically Bengali, Odia, and Gujarati. The primary objective was to address challenges related to data scarcity, multilingual understanding, and efficient model adaptation using existing pretrained language models.

A key contribution of this work is the implementation and evaluation of a translation-based inference pipeline, where queries in Indic languages are translated into English, processed using a pretrained language model, and translated back into the original language. The experimental results demonstrate that this approach significantly improves response quality, fluency, and coherence compared to direct inference in low-resource languages. This highlights the strong dependence of current language models on high-resource languages such as English.

However, the study also reveals important limitations. The multi-stage pipeline introduces increased latency, making real-time deployment challenging. Additionally, the system does not fully preserve linguistic nuances, and translation errors can propagate through the pipeline, sometimes leading to outputs that deviate from the original query intent.

Overall, this work demonstrates that translation-based pipelines provide an effective and practical solution for extending LLM capabilities to low-resource languages, especially in resource-constrained settings. At the same time, it underscores the need for more native support for Indic languages in language models.

7.2. Future Work

While the current approach provides a functional baseline, several directions can be explored to improve performance and scalability.

First, future work will focus on developing a dedicated Small Language Model trained directly on Indic language corpora. This would reduce reliance on translation and enable better handling of linguistic nuances and cultural context. Leveraging datasets from sources such as AIKosh and AI4Bharat, along with synthetic data generation techniques, can help address the issue of limited labeled data.

Second, improvements in the data pipeline can further enhance model performance. This includes better language filtering, advanced data cleaning techniques, and more effective tokenization strategies tailored to Indic scripts.

Third, optimization of the inference pipeline is necessary to reduce latency. Techniques such as model quantization, smaller architectures, and efficient decoding strategies can make the system more suitable for real-time applications.

Additionally, incorporating high-quality multilingual translation models or jointly training models on both Indic and English data can improve cross-lingual alignment and reduce translation errors.

Finally, future research can explore instruction tuning and alignment techniques specifically for Indic languages, enabling the model to better follow user queries and generate more contextually relevant responses. Expanding evaluation to include more languages and standardized benchmarks will also provide a more comprehensive understanding of system performance.

Work Update

8. Work Update

8.1 Work Completed

The work completed so far primarily focuses on building a strong theoretical and practical foundation required for the project, along with designing the initial experimental approach.

Background Study and Preparation

A comprehensive background study was conducted covering key areas of deep learning and modern language models:

- **PyTorch Framework:** Gained hands-on experience with model building, tensor operations, backpropagation, and optimization techniques. This serves as the primary tool for implementing and experimenting with models.
- **Core Neural Network Concepts:**
    - Activation functions (ReLU and variants) for introducing non-linearity
    - Feedforward Neural Networks (FFNs) as core components of transformer architectures
    - Convolutional Neural Networks (CNNs) for understanding alternative architectures
- **Attention Mechanism:**
    - Studied self-attention and its ability to capture relationships between all tokens in a sequence
    - Understood how attention assigns importance weights to generate context-aware representations
- **Transformer Architecture:**
    - In-depth study of “Attention Is All You Need”
    - Key components: Multi-head attention, encoder-decoder structure, positional encoding, and FFNs
    - Understanding transformers as the backbone of modern LLMs
- **Research Papers Reviewed:**
    - Attention Is All You Need – foundational architecture for LLMs
    - Survey on Model Compression for LLMs – techniques like pruning, quantization, and distillation for efficient models

This background study directly supports the project’s goal of working with small and efficient language models, especially for Indic languages.

Experimental Design: Translation-Based LLM Pipeline

A key component of the work completed is the design and conceptualization of the translation-based pipeline, which serves as the core experiment of the project.

- The approach follows a pivot-based multilingual inference strategy:
    - Input in a low-resource Indic language is translated to English
    - The translated text is processed using an LLM (e.g., Gemma, LLaMA)
    - The generated response is translated back to the original language
- This method allows leveraging the strong reasoning and generation capabilities of English-trained LLMs without requiring large-scale Indic datasets.
- The experiment is planned to be conducted on multiple models (Gemma, LLaMA, etc.) to compare:
    - Response quality
    - Consistency across languages
    - Sensitivity to translation errors
- The approach is particularly relevant because:
    - Indic languages are often low-resource
    - Training native models is data and compute intensive
    - Translation-based pipelines provide a scalable and practical alternative
- Initial analysis also considers potential limitations:
    - Error propagation due to imperfect translation
    - Loss of cultural or contextual nuances
    - Dependency on external translation systems

This experimental setup forms a strong starting point for the project, enabling practical evaluation of multilingual capabilities without heavy training requirements.

8.2 Work Remaining

The primary objective of the remaining work is to move beyond the translation-based pipeline and develop a small, efficient open-source language model for low-resource Indic languages. The focus will be on training, evaluating, and fine-tuning models for Bengali, Odia, and Gujarati.

1. Dataset Collection and Preparation

The first step involves building a high-quality multilingual dataset from diverse sources:

- Curated and Government Datasets:
    - AIKosh
    - AI4Bharat datasets
    - HuggingFace Indic datasets
    - OSCAR corpus

- Structured Unlabelled Data:
    - Wikipedia (for grammatically correct text)
    - News sources:
        - Anandabazar Patrika (Bengali)
        - Sambad (Odia)
        - Gujarat Samachar
- Synthetic Data Generation:
    - Generating question-answer pairs using stronger LLMs
    - Creating translated data to augment low-resource corpora

2. Data Pipeline Development

A robust data pipeline will be implemented to ensure dataset quality and usability:

- Language Filtering:
    - Using tools like LangDetect to retain only target languages
    - Removing mixed-language and noisy text
- Data Cleaning:
    - Removing HTML tags and unwanted symbols
    - Deduplication of repeated content
    - Filtering low-quality or irrelevant text
- Data Categorization:
    - Pretraining corpus (raw text)
    - Instruction/QA datasets
    - Translation pairs
- Tokenization:
    - Using SentencePiece or BPE tokenization
    - Handling Unicode and script-specific challenges for Indic languages

3. Model Training Strategy

The next phase involves training and experimenting with open-source LLM frameworks and models:

- Framework:
    - OLMo (open-source LLM training framework)
- Model Architectures:
    - LLaMA
    - Gemma
    - Sarvam AI models
- Training Stages:
    - Pretraining:

        - Using large-scale corpora (OSCAR, AI4Bharat, AIKosh)
        - Goal: Learn general language understanding
    - Initial Evaluation:
        - Assessing performance on basic language tasks
        - Determining feasibility and effectiveness
    - Instruction Tuning:
        - Fine-tuning on QA datasets, translation tasks, and synthetic conversations
        - Goal: Make the model conversational and task-oriented

4. Evaluation and Iteration

- Evaluate model performance across:
    - Language understanding
    - Response generation quality
    - Cross-lingual consistency
- Based on initial results:
    - Improve dataset quality
    - Refine training strategies
    - Optimize model size and efficiency

5. Long-Term Goal

If initial training results are promising, the project will progress toward:

- Building a fully conversational Indic small language model
- Reducing dependence on translation-based pipelines
- Creating an efficient model suitable for real-world deployment in low-resource settings

This phase represents the transition from experimental validation to actual model development, forming the core contribution of the project.

References

9. References

9.1 Research Papers

- Vaswani, A., et al. (2017). *Attention Is All You Need*.
  Available at:
  https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- *A Survey on Model Compression for Large Language Models*
- Brown, T. B., et al. (2020). *Language Models are Few-Shot Multilingual Learners*
- *On the Multilingual Capabilities of Large Language Models*

9.2 Articles and Conceptual Resources

- Microsoft Azure. *What are Small Language Models?*
  https://azure.microsoft.com/en-in/resources/cloud-computing-dictionary/what-are-small-language-models
- Hugging Face Blog. *Small Language Models*
  https://huggingface.co/blog/jjokah/small-language-model
- IBM. *Small Language Models Explained*
  https://www.ibm.com/think/topics/small-language-models

9.3 Learning Resources (Videos & Tutorials)

PyTorch

- https://youtu.be/c36lUUr864M

Attention Mechanism

- https://youtu.be/bCz4OMemCcA

Overview of LLMs

- https://youtu.be/qahhpC_NEFY
- https://youtu.be/eMlx5fFNoYc?list=PLyVzKGKqOBMTeM8q21jiZnd2R9a-S2RKg
- https://youtu.be/9-Jl0dxWQs8?list=PLyVzKGKqOBMTeM8q21jiZnd2R9a-S2RKg

Detailed NLP Course

- https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

9.4 Project Resources and Work Links

- **GitHub Repository**: https://github.com/Levixon10/SLMS_IndicLanguages_SOP
- **Content Drive**: https://drive.google.com/drive/u/1/folders/1lFZH9F173AGWNDpOUYjPgvAqswWjY0dO
- **Weekly Progress Tracker**: https://docs.google.com/spreadsheets/d/1tMId9dc9xpMZKLzYGPemuNoKt76AToOABpkQu0eXqdM/edit?usp=sharing
- **Base Model (Colab Notebook)**: https://colab.research.google.com/drive/1PrL_3bxnUn2db34jF-rQAwSb8FjZtV-i?usp=sharing
- https://colab.research.google.com/drive/1f69oJPt_9tZqC1WSwKf1C7EN4CbpxIjH